

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

树的常见算法

Common Tree Algorithms: Depth, Size, Centroid, Diameter and LCA

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 30 日

目录

1. 树的基础
2. 深度与子树大小
3. 树的重心
4. 树的直径
5. 最近公共祖先
6. 总结

目录

1 树的基础

2 深度与子树大小

3 树的重心

4 树的直径

5 最近公共祖先

6 总结

什么是树

树的定义：n 个节点的连通无环无向图。

三条等价性质

- 恰好有 $n-1$ 条边
- 任意两点之间有且只有一条路径
- 去掉任意一条边就不再连通

有根树：指定一个节点为根，其余节点分成父节点与子节点，形成清晰的层级。

有根树把树从图变成层级结构

树的术语

示例树：根为节点 1，父子关系如下。

- 节点 1 的子节点：2, 3
- 节点 2 的子节点：4, 5
- 节点 3 的子节点：6

基本术语

- 父节点：节点 4 的父节点是 2
- 深度：根深度为 0，节点 4 的深度为 2
- 子树：节点 2 的子树是 $\{2, 4, 5\}$
- 叶子：没有子节点的节点，如 4, 5, 6

深度从上往下数，子树从节点往下取

树的存储：邻接表

常用存储：邻接表，每个节点保存一张邻居列表。

加边

- 无向边 (u, v)：在 $g[u]$ 中加 v，同时在 $g[v]$ 中加 u
- 有根树也可以只存子节点，省一半空间

遍历方式

- DFS：递归进入子节点，回来处理当前节点
- BFS：按层扩展，适合求深度与最短距离

关键细节：遍历时传入父节点，遇到父节点就跳过，否则会在无向边上来回死循环。

树是无向图，遍历必须带父节点

树的 DFS 通用框架

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
vector<int> g[MAXN];
void dfs(int u, int fa) {
    // 进入节点 u, fa 是父节点
    for (int v : g[u]) {
        if (v == fa) continue;     // 跳过父节点
        dfs(v, u);                 // 递归处理子节点
    }
    // 离开节点 u
}
```

目录

1 树的基础

2 深度与子树大小

3 树的重心

4 树的直径

5 最近公共祖先

6 总结

例题：深度与子树大小

题目：给定一棵以节点 1 为根的树，求每个节点的深度 $dep[u]$ 和子树大小 $sz[u]$。

示例

- **父节点关系**：$fa[2] = fa[3] = 1$，$fa[4] = fa[5] = 2$，$fa[6] = 3$
- **深度**：$dep[1] = 0$，$dep[2] = 1$，$dep[4] = 2$
- **子树大小**：$sz[4] = 1$，$sz[2] = 3$，$sz[1] = 6$

深度与子树大小：解法

核心思路：一次 DFS 解决两件事——往下走时传递深度，回来时累加子树大小。

关键步骤

- 进入节点 u，先把 $sz[u]$ 初始化为 1（自己算一个）
- 递归进入子节点前，$dep[v] = dep[u] + 1$，把深度往下传
- 子节点返回后，$sz[u] += sz[v]$，把子树累加上来
- 根节点深度为 0，其余全部沿父链自然累加

为什么正确：深度只依赖父节点，一条链走下来就是对的；子树大小 = 自己 + 所有子节点的子树大小，回溯正好累加一遍。

一次 DFS 同时算出深度和子树大小

深度与子树大小代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
int dep[MAXN], sz[MAXN];
void dfs(int u, int fa) {
    sz[u] = 1;                          // 子树大小初值为 1
    for (int v : g[u]) {
        if (v == fa) continue;
        dep[v] = dep[u] + 1;            // 深度向下传递
        dfs(v, u);
        sz[u] += sz[v];                 // 回溯累加
    }
}
```

目录

1 树的基础

2 深度与子树大小

3 树的重心

4 树的直径

5 最近公共祖先

6 总结

例题：树的重心

题目：给定一棵树，求它的重心。重心定义为：删去该节点后，剩余各连通块中节点数最多的那一块最小。

示例

- 路径 $1-2-3-4$：重心是 2 或 3，删去后最大块为 2
- 星形 1 连 $2,3,4,5$：重心是 1，删去后最大块为 1
- 一棵树可能有 1 个或 2 个重心

树的重心：解法

核心思路：枚举删点，找出最大连通块最小的那个点。

关键步骤

- 先一次 DFS 求出所有节点的 sz
- 对每个 u，计算删掉它之后的最大连通块：$\max(\max_{v \in son} sz[v],\ n - sz[u])$
- 全局取最小的那个最大块，对应的 u 就是重心
- 根节点没有上方部分，$n - sz[root] = 0$

为什么正确：删掉 u 后，树分成" 每个子节点的子树" 和"u 上方剩余" 两类连通块，最大块一定是这两类里最大的那个。取所有点中最小的最大块，即符合重心定义。

枚举删点，最小化最大连通块

树的重心代码


```
int sz[MAXN];
int N;
int centroid, best;
void dfs(int u, int fa) {
    sz[u] = 1;
    int mx = 0;
    for (int v : g[u]) {
        if (v == fa) continue;
        dfs(v, u);
        sz[u] += sz[v];
        mx = max(mx, sz[v]);          // 子节点中最大的子树
    }
    mx = max(mx, N - sz[u]);          // 加上方部分
    if (mx < best) {
        best = mx;
        centroid = u;
    }
}
```

目录

1 树的基础

2 深度与子树大小

3 树的重心

4 树的直径

5 最近公共祖先

6 总结



例题：树的直径

题目：给定一棵树，求树的直径，即任意两点间距离的最大值。

示例

- 路径 $1-2-3-4$：直径 3，端点为 1 和 4
- 星形 1 连 $2,3,4,5$：直径 2，端点为任意两个叶子
- 树 $\{1-2,\ 2-4,\ 2-5,\ 1-3,\ 3-6\}$：直径 4，端点为 4 和 6

树的直径：解法

核心思路：两次 DFS，先从任意点找最远点 A，再从 A 找最远点 B，AB 就是直径。

关键步骤

- 第一次 DFS：从任意节点出发，找到离它最远的点 A
- 第二次 DFS：从 A 出发，找到离 A 最远的点 B
- A 到 B 的距离即为直径；只有一个节点时直径为 0

为什么正确：树上从任意点出发，最远的点一定是某条直径的端点，所以第一次找到的 A 就是直径一端；从 A 再找最远点，B 就是另一端。该结论只对树成立，带环图不适用。

复杂度：两次 DFS，总时间 $O(n)$。

两次 DFS 找两端点

树的直径代码


```cpp
int far_node, far_dist;
void dfs(int u, int fa, int d) {
    if (d > far_dist) {
        far_dist = d;
        far_node = u;
    }
    for (int v : g[u]) {
        if (v == fa) continue;
        dfs(v, u, d + 1);
    }
}
int tree_diameter() {
    far_dist = -1;
    dfs(1, 0, 0);
    int A = far_node;
    far_dist = -1;
    dfs(A, 0, 0);
    return far_dist;
```

树的直径代码


```
19 }
```

目录

1 树的基础

2 深度与子树大小

3 树的重心

4 树的直径

5 最近公共祖先

6 总结

例题：最近公共祖先

题目：给定一棵以节点 1 为根的树，多次询问：节点 u 和 v 的最近公共祖先（LCA）是谁。

示例

- **父节点关系**：$fa[2] = fa[3] = 1$，$fa[4] = fa[5] = 2$，$fa[6] = 3$
- $u = 4$，$v = 5$，LCA 为 2
- $u = 4$，$v = 6$，LCA 为 1
- $u = 2$，$v = 2$，LCA 为 2

最近公共祖先：朴素解法

核心思路：先对齐深度，再一起向上跳，直到相遇。

关键步骤

- 预处理：DFS 一次，求出每个节点的 dep 和 fa
- 让较深的那个沿 fa 上跳，跳到与浅的同深度
- 此时若两点重合，该点就是 LCA，直接返回
- 否则两者同步沿 fa 上跳，直到相遇，相遇点就是 LCA

为什么正确：LCA 的深度不超过两点中较浅的那个，所以先对齐深度不会跳过；同步上跳时，第一个相遇的点就是最深的公共祖先，即 LCA。

复杂度与优化：单次 $O(n)$，树越深越慢；倍增法可降到 $O(\log n)$，本课不展开。

先对齐深度，再一起向上跳

最近公共祖先代码


```cpp
int fa[MAXN], dep[MAXN];
void dfs(int u, int p, int d) {
    fa[u] = p;
    dep[u] = d;
    for (int v : g[u]) {
        if (v == p) continue;
        dfs(v, u, d + 1);
    }
}
int lca(int u, int v) {
    while (dep[u] > dep[v]) u = fa[u];    // 对齐深度
    while (dep[v] > dep[u]) v = fa[v];
    while (u != v) {                      // 同步上跳
        u = fa[u];
        v = fa[v];
    }
    return u;
}
```

目录

1 树的基础

2 深度与子树大小

3 树的重心

4 树的直径

5 最近公共祖先

6 总结

总结（一）：本课六个基本量

一棵树上的六种基础信息

- 深度：从根往下数，DFS 一次求出
- 子树大小：自己 1 加上所有子节点子树大小
- 重心：删点后最大连通块最小的节点
- 直径：两次 DFS 找两端点
- LCA：先对齐深度，再同步上跳

所有量都能靠一次或两次 DFS 求出

总结（二）：DFS 模板通用性

量	转移	依赖
深度	$dep[v] = dep[u] + 1$	父节点
子树大小	$sz[u] = 1 + \sum sz[v]$	子节点
重心	删点后最大块最小	子树大小
直径	两次 DFS 找端点	最远点
LCA	对齐深度 + 同步上跳	dep 与 fa

易错点

- 遍历时要传父节点，否则无向边上来回死循环
- 根节点的深度和父节点要单独处理
- 重心可能有两个，代码返回其中一个即可

总结（三）：树的算法路线图

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

本课覆盖的层次

- 基础：DFS 遍历、邻接表存储
- 静态信息：深度、子树大小
- 全局信息：重心、直径
- 路径查询：LCA 朴素版

进一步学习方向

- 树上差分、DFS 序与树状数组
- 倍增 LCA、树链剖分、Tarjan LCA
- 树形 DP：换根 DP、背包 DP

DFS 是树的万金油，先吃透它

附录：常见陷阱

遍历

- 忘记传父节点，递归死循环或栈溢出
- 用 BFS 时队列元素要记父节点

深度与子树

- 根节点深度是 0 还是 1，全文要统一
- 子树大小初值必须是 1，不能是 0

直径

- 两次 DFS 法只对树成立，一般图会出错
- 树上有负权边时，要用树形 DP 而不是两次 DFS

LCA

- 朴素法深度差很大时很慢，可用倍增优化